

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1007	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	RISHIK JK	Reg. No.:	192511052
Date:	13/08/2026	Duration:	60 minutes

Code of Conduct

I **RISHIK JK (192511052)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **RISHIK JK**

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

`CO3-AT1-GIT & DOCKER TECHNICAL ASSIGNMENT

NAME: JK RISHIK

SLOT: D

REGNO :192511052

COURSE CODE: CSA1007

DATE:11/08/2026

1. Problem Analysis

Problem Statement

The Smart Renewable Battery Recycling Platform is designed to provide a centralized system for managing the collection, storage, transportation, and recycling of used batteries. The platform helps recycling organizations monitor battery inventory, track recycling activities, identify hazardous battery conditions, manage transportation, and generate sustainability reports. By organizing these activities in a single platform, the system aims to improve recycling efficiency, operational safety, and sustainable battery management.

Project Objectives

- To monitor battery inventory and storage conditions effectively.
- To manage and optimize battery recycling activities.
- To identify hazardous battery conditions at an early stage.
- To track battery collection and transportation activities.
- To generate useful recycling and sustainability reports.
- To provide timely notifications for important battery conditions.

Functional Requirements

The system provides the following major functions:

- User authentication and access management.
- Battery inventory management.
- Battery condition monitoring.
- Hazard detection and notification.
- Recycling operation management.
- Collection and transportation tracking.
- Sustainability report generation.
- Monitoring dashboard for recycling activities.

Expected Outcomes

- Improved battery recycling efficiency.
- Reduced environmental impact from battery waste.
- Better recovery of useful battery materials.
- Improved safety during battery storage and handling.
- Better monitoring of recycling activities.
- More organized and sustainable battery management.

2. Project Structure Design

The project repository is organized into separate folders and files so that the application code, database components, testing files, and deployment configuration can be maintained independently. A well-organized repository makes the project easier to develop, test, maintain, and manage using Git.

Project Components

- **app/** – Contains the main application components of the system.
- **routes/** – Handles different application requests and system navigation.
- **services/** – Contains the main business logic for inventory, recycling, logistics, and notifications.
- **models/** – Manages the application data and database models.
- **templates/** – Contains the HTML pages used for the user interface.
- **static/** – Contains CSS, JavaScript, and other frontend resources.
- **database/** – Contains the database structure and related SQL files.
- **tests/** – Contains test files used to check the functionality of the system.
- **Dockerfile** – Defines the environment required to run the application inside a Docker container.
- **docker-compose.yml** – Defines the application and database services used by Docker.
- **requirements.txt** – Contains the Python packages required by the application.
- **.gitignore** – Prevents unnecessary or sensitive files from being added to the Git repository.
- **README.md** – Provides information about the project, installation, and usage.
- **app.py** – Acts as the main entry point of the application.

3. Git Repository Initialization

Git is used as the version control system for the Smart Renewable Battery Recycling Platform. It maintains the history of project development and allows changes made by different team members to be tracked systematically.

The project is maintained in a Git repository so that different versions of the source code can be stored and managed. It also supports collaboration by allowing developers to work on separate features without affecting the stable version of the project.

Essential Git Components

- **Repository:** Stores the complete project source code and its version history.
- **Working Directory:** Contains the current project files being developed.
- **Staging Area:** Holds changes selected for the next version.
- **Commit History:** Maintains records of changes made to the project.
- **Remote Repository:** Provides a shared location for team members to synchronize their work.
- **.gitignore:** Prevents unnecessary or sensitive files from being included in the repository.

4. Git Branching Strategy

A **feature-based branching strategy** is suitable for the Smart Renewable Battery Recycling Platform because the project contains multiple independent functional areas such as inventory management, recycling management, logistics, and hazard notifications.

The stable version of the project is maintained separately from ongoing development. Individual features can be developed independently and integrated after testing.

Branches Used

- **Main Branch** – Maintains the stable and tested version of the project.
- **Develop Branch** – Contains the latest development version of the project.
- **Inventory Feature Branch** – Used for developing battery inventory management.
- **Recycling Feature Branch** – Used for developing recycling operation management.
- **Logistics Feature Branch** – Used for developing battery transportation and logistics tracking.
- **Alert Feature Branch** – Used for developing hazardous condition alerts and notifications.

Justification

This branching strategy supports parallel development and reduces the possibility of affecting the stable application. Each feature can be developed and tested independently before being integrated into the development version. This approach also makes project maintenance and future enhancement easier.

5. Version Control Workflow

Git provides an organized workflow for managing the development of the project. Each feature is developed separately and changes are recorded throughout the development process.

The main activities in the workflow are:

- Create a separate branch for each feature.
- Develop and test the feature independently.
- Record meaningful changes in the project history.
- Review completed changes before integration.
- Merge tested features into the development branch.
- Maintain the stable version separately.
- Synchronize the project with the remote repository.

Conflict Resolution

A merge conflict may occur when different team members modify the same part of a project file. The conflicting changes are reviewed and the correct version is selected. After resolving the conflict, the corrected version is saved and integrated into the project.

This process helps maintain consistency and prevents accidental loss of important project changes.

6. Commit History Analysis

Commit history provides a chronological record of the development activities performed in the project. Meaningful commit messages make it easier to understand the purpose of each change.

Example Commit Activities

- Initial project setup.
- Creation of project structure.
- Development of battery inventory management.
- Development of recycling management.
- Development of logistics tracking.
- Implementation of hazard notifications.
- Integration of database functionality.
- Addition of Docker configuration.
- Testing and bug fixing.
- Documentation updates.

Good commit practices improve project maintenance because developers can easily identify previous changes and understand how the project has developed over time.

7. Docker Environment Design

Docker is suitable for the Smart Renewable Battery Recycling Platform because it provides a consistent environment for development, testing, and deployment. It packages the application and its dependencies into an isolated environment.

The proposed Docker environment contains two major components:

Application Container

The application container runs the web application and manages battery inventory, recycling activities, logistics, notifications, and other system functions.

Database Container

The database container stores user details, battery records, recycling information, transportation records, and sustainability data.

Benefits of Docker

- Provides a consistent development environment.
- Simplifies application deployment.
- Improves portability across different systems.
- Separates application and database services.
- Makes testing and maintenance easier.
- Supports future scalability.

8. Docker file Development

A Docker file defines the environment required to run the Smart Renewable Battery Recycling Platform. It specifies the base environment, application dependencies, project files, application port, and startup configuration.

Main Docker file Components

- **Base Image:** Provides the required Python environment.
- **Working Directory:** Defines where the application runs inside the container.
- **Dependencies:** Installs the packages required by the application.
- **Application Files:** Places the project files inside the container.
- **Application Port:** Defines the port used by the web application.
- **Startup Configuration:** Specifies how the application starts.

The Docker file makes the application portable and allows the same application environment to be used during development, testing, and deployment.

9. Docker Compose Design

Docker Compose is used to manage the different services required by the Smart Renewable Battery Recycling Platform. It allows the application and database to run together in a properly connected environment.

Main Components

- **Application Service** – Runs the Smart Renewable Battery Recycling Platform and handles the main application functions.
- **Database Service** – Stores user details, battery information, recycling records, logistics details, and sustainability data.
- **Network** – Allows the application and database services to communicate with each other.
- **Environment Variables** – Store important configuration details such as database name, username, and connection information.
- **Volume** – Stores database information safely so that the data is not lost when the database container is restarted.
- **Service Dependency** – Ensures that the application is configured to work with the required database service.

Advantages of Docker Compose

- It simplifies the management of multiple services.
- It provides a consistent development and deployment environment.
- It makes application and database configuration easier.
- It supports easy testing and deployment of the complete system.

10. Container Deployment and Testing

The application is deployed using the Docker environment designed for the project. The application and database services are started together and tested to verify that they work correctly.

Testing Activities

- Check whether the application is running correctly.
- Check whether the database is available.
- Verify application and database communication.
- Test user authentication.
- Test battery inventory management.
- Test battery condition monitoring.
- Test recycling operation management.
- Test logistics tracking.
- Test hazard notifications.
- Test sustainability report generation.
- Verify that stored data remains available after restarting the application.

Expected Result

The application should run successfully in the Docker environment and communicate correctly with the database. All major functions should work without critical errors.

11. Benefits of Git and Docker

Git and Docker provide useful support for developing and managing the Smart Renewable Battery Recycling Platform. Git mainly helps with version control and teamwork, while Docker helps with application deployment and portability.

Benefits of Git

- **Collaboration** – Team members can work on different features without affecting each other's work.
- **Version Management** – All important changes made to the project can be tracked and maintained.
- **Branching** – Different features can be developed separately and integrated after testing.
- **Backup** – The project source code can be stored safely in a remote repository.
- **Maintenance** – Previous versions can be reviewed when errors or problems occur.

Benefits of Docker

- **Portability** – The application can run consistently on different systems.
- **Easy Deployment** – The application environment can be prepared and deployed easily.
- **Consistency** – The same software environment can be used during development and testing.
- **Isolation** – The application and database can run in separate containers.
- **Maintainability** – Individual services can be managed and updated independently.
- **Scalability** – Additional containers can be added when the system grows.